

GitOps

O que você aprenderá nesta página

Comparar GitOps a outros métodos de implantação (*deployment*) Comparar a implantação tradicional baseada em *push* (*push deployment*) com a implantação baseada em *pull* (*pull deployment*) Implementar GitOps no seu cluster Kubernetes com o ArgoCD

Um pipeline de implantação simples padrão inclui as seguintes etapas:

O desenvolvedor envia (*pushes*) o código modificado para um repositório, como o GitHub. Isso aciona um serviço de CI/CD, como o GitHub Actions, para iniciar a execução. O serviço de CI/CD executa os testes, compila uma imagem, envia a imagem para um registro e implanta a nova imagem, por exemplo, no Kubernetes.

Isso é chamado de **implantação baseada em push** (*push deployment*). O nome é descritivo, pois tudo é "empurrado" (*pushed*) para a frente pelo passo anterior. Existem alguns desafios com a abordagem de *push*. Por exemplo, se tivermos um cluster Kubernetes que não está disponível para conexões externas, ou seja, o cluster na sua máquina local ou qualquer cluster ao qual não queremos dar acesso a terceiros. Nesses casos, não é possível fazer com que o CI/CD envie a atualização para o cluster.

Na **configuração de pull** (*pull configuration*), o processo é revertido. O cluster, rodando em qualquer lugar, **busca** (*pull*) a nova imagem e a implantação ocorre automaticamente. A nova imagem ainda será testada e compilada pelo CI/CD. Nós simplesmente aliviamos o CI/CD do fardo da implantação e transferimos a responsabilidade de "buscar" os dados (*pulling*) para um sistema separado.

Watchtower

Se você concluiu o curso de DevOps com Docker, deve ter ouvido falar sobre o Watchtower, que nos permitiu transformar os "pushes" finais do pipeline de implantação em "pulls" quando rodamos serviços com docker-compose.

GitOps trata-se exatamente desta inversão, promovendo boas práticas para o lado operacional. Isto é alcançado mantendo o estado do cluster em um repositório Git, gerenciando, portanto, não só implantações de aplicações, mas também todas as mudanças de infraestrutura no cluster. Isso exigirá configuração adicional e uma mudança de mentalidade, superando a tradição de configurações convencionais em servidores. Mas, quando chegarmos a esse ponto, o GitOps será o último prego no caixão do gerenciamento imperativo de clusters.

O ArgoCD é atualmente a principal ferramenta para GitOps em Kubernetes, e é a nossa escolha para este módulo. No fim das contas, nosso fluxo de trabalho deve ficar assim:

O desenvolvedor executa *git push* com código ou configurações modificadas. O serviço de CI/CD (GitHub Actions, no nosso caso) inicia a execução. O serviço de CI/CD compila e faz o *push* da nova imagem e confirma a alteração na ramificação principal (*main*, no nosso caso). O ArgoCD obterá o estado descrito na *branch* de release (ramo de liberação) e configurará esse estado no nosso cluster.

Vamos iniciar instalando o ArgoCD:

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

Agora o ArgoCD está sendo executado no nosso cluster. Ainda precisamos abrir o acesso a ele. Existem várias opções; usaremos um LoadBalancer:

```
kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "LoadBalancer"}}'
```

A senha inicial para a conta *admin* é gerada automaticamente e armazenada em texto claro no campo *password* na *secret* chamada *argocd-initial-admin-secret*. Decodificamos a string em base64 com este comando:

```
kubectl get -n argocd secrets argocd-initial-admin-secret -o yaml
```

Quando sincronizarmos novamente as mudanças no ArgoCD, um novo pod funcional iniciará e nosso app passará a rodar!

Se mudarmos a política de sincronização de manual para **automática** na interface do Argo, qualquer mudança no GitHub será aplicada automaticamente no cluster. Teste aumentar o número de réplicas no *deployment.yaml* e faça o commit; o ArgoCD irá sincronizar as mudanças por conta própria (frequência de checagem a cada 180 segundos) e escalar a aplicação.

O repositório Git (o *source*) torna-se a única fonte da verdade, contendo o *deployment.yaml*, *service.yaml* e o *kustomization.yaml*. Qualquer edição à imagem da aplicação passará primeiramente pelas ferramentas do Kustomize e pelo pipeline no GitHub Action.

Para que isso funcione, também é necessário permitir permissões de gravação de repositório (*write permissions*) na configuração de fluxo de trabalho (*workflow*) do GitHub. E agora temos uma integração GitOps 100% automatizada!

Os benefícios da implantação com GitOps incluem:

Melhor Segurança: Ninguém precisa de acesso ao cluster, nem os serviços de CI/CD. Não há necessidade de compartilhar chaves com terceiros; todos apenas confirmam (*commit*) o código no Git. **Maior Transparência:** Todo o escopo é definido no GitHub. Se uma pessoa nova entrar no time, ela só precisará verificar o repositório, não sendo necessário repassar conhecimentos ocultos. **Maior Rastreabilidade:** Todas as mudanças no cluster têm controle de versão. É fácil saber quem fez as mudanças e como ficou o estado. **Redução de Risco:** Caso ocorra algum erro e as coisas quebrem, usar um comando `git revert` é o suficiente para voltar à normalidade do cluster. **Portabilidade:** Ao mudar de provedor de Nuvem, basta montar um novo cluster e apontá-lo para o mesmo repositório - pronto, seu cluster está implementado novamente.

Mova o aplicativo *Log output* para usar o GitOps para que, ao fazer o *commit* no repositório, o aplicativo seja atualizado automaticamente.

Mova O Projeto (*The Project*) para usar o GitOps de maneira que, após um *commit*, a aplicação seja atualizada automaticamente. Neste exercício, basta que a *branch main* (ramificação principal) faça o *deploy* no cluster.

Múltiplos ambientes (More environments)

Se um aplicativo possuir muitos ambientes, como produção (*production*), teste (*staging*) e desenvolvimento, defini-los de forma direta envolve muito copia-e-cola (*copy-paste*). Com o uso do Kustomize, a recomendação é definir uma "base" (*base*) comum para todos e em seguida utilizar as "camadas" (*overlays*) relativas a cada ambiente, onde apenas as partes diferentes estarão configuradas.

As partes fixas permanecem em base, já em `overlays/prod/deployment.yaml` alteramos por exemplo, as réplicas.

Definindo o todo com YAML (Defining the whole with yaml)

Até agora usamos a UI do ArgoCD para definir *applications*. Porém, isso também pode ser feito de modo declarativo e definindo uma configuração como um recurso do Kubernetes.

Depois de declarar tudo num único YAML, apenas integre via `kubectl apply -n argocd -f application.yaml`.

Melhore as definições do seu Projeto da seguinte forma:

Crie dois ambientes separados (*production* e *staging*) em *namespaces* diferentes. Cada commit na ramificação *main* deve fazer o *deploy* no *staging*. Cada commit **com tag** deve fazer o *deploy* no ambiente *production*. Em *staging*, o *broadcaster* apenas registra mensagens (logs); não deverá enviá-las para serviços externos. Em *staging*, o banco de dados não recebe *backups*.

Finalize a infraestrutura do seu projeto, fazendo com que sejam utilizados repositórios totalmente distintos: um para os códigos-fonte da aplicação e outro unicamente para as configurações de *deployment*.